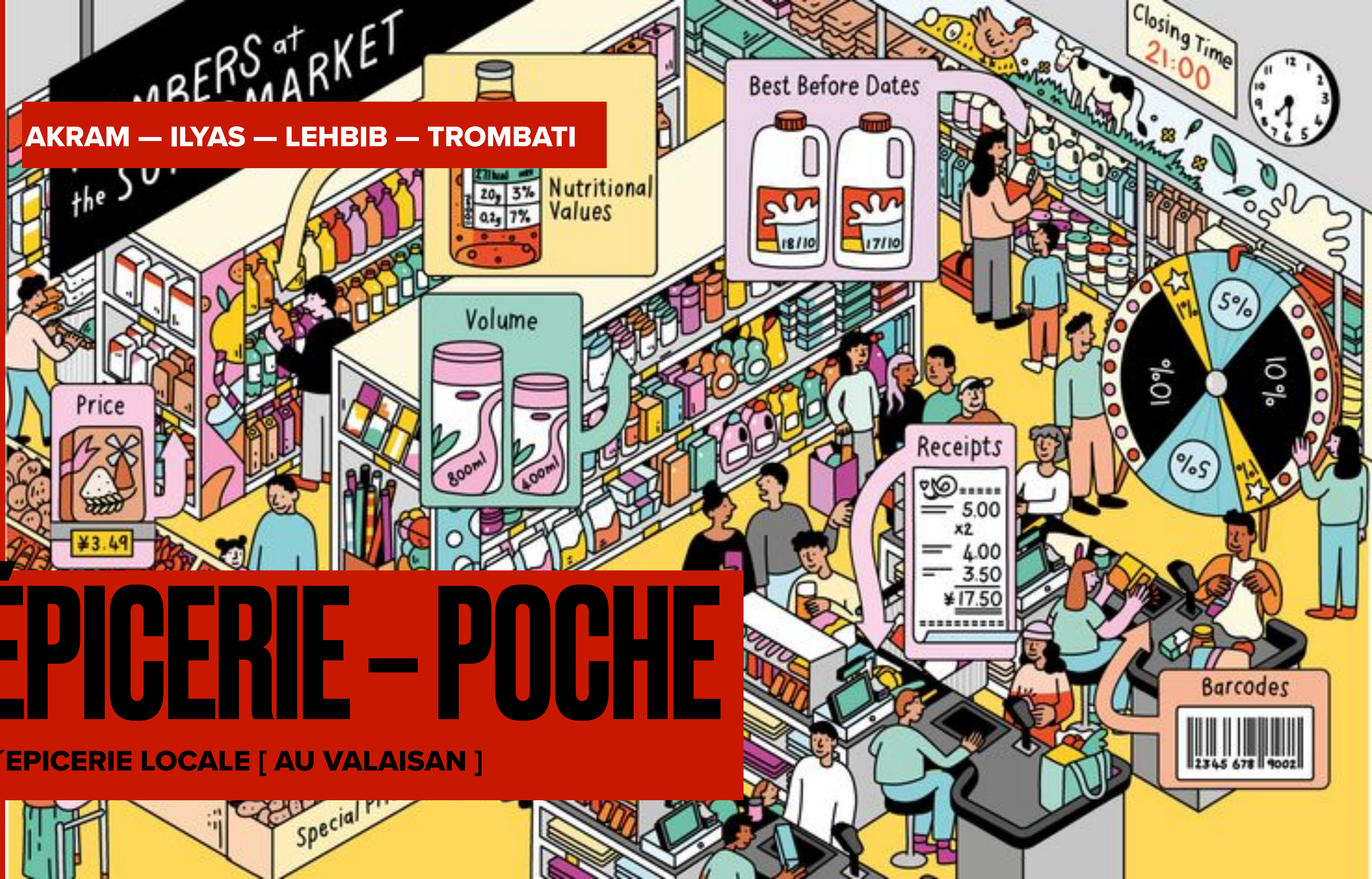


AKRAM — ILYAS — LEHBIB — TROMBATI

ÉPICERIE - POCHE

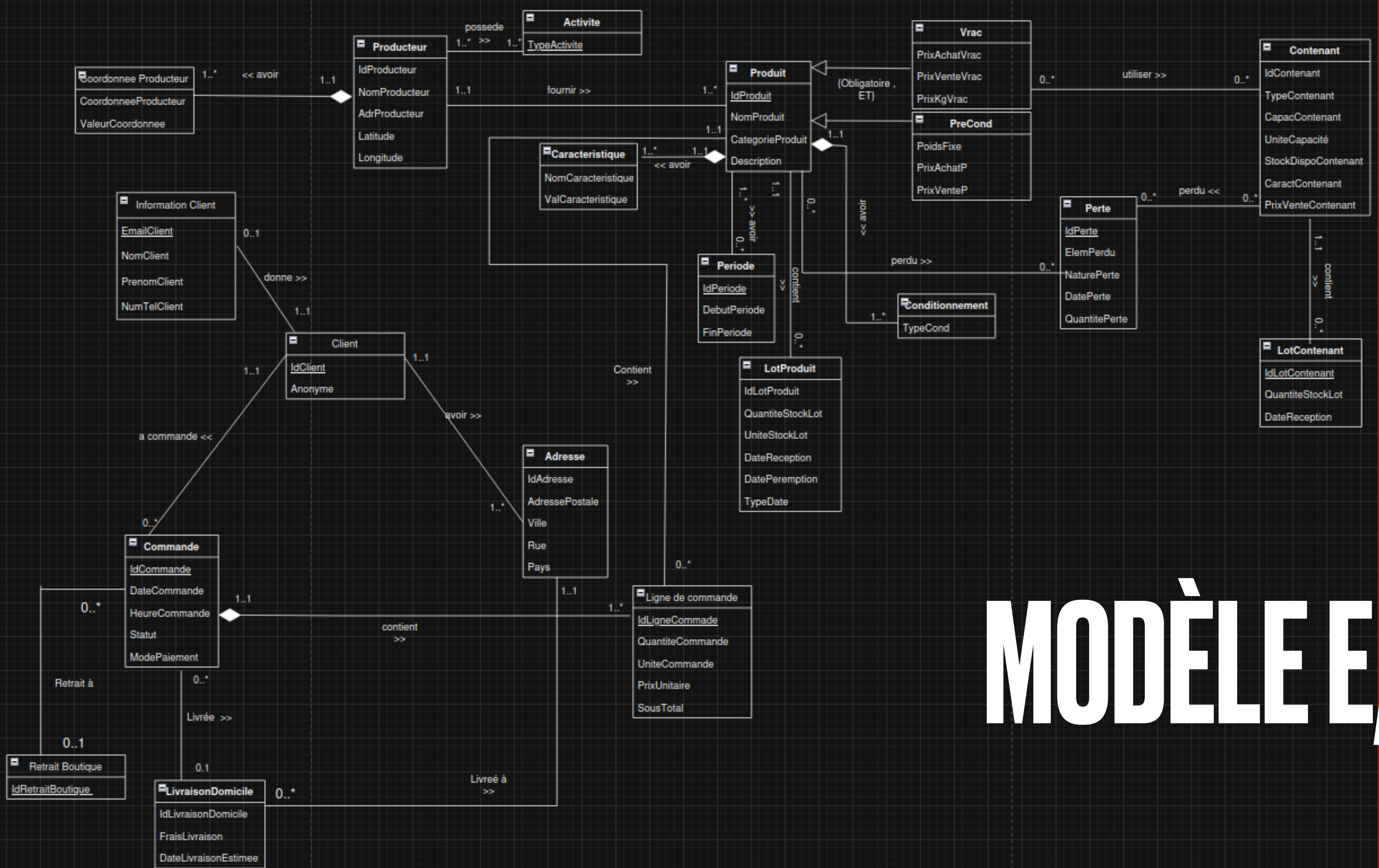
L'ÉPICERIE LOCALE [AU VALAISAN]



ANALYSE DU PROBLÈME

- *Produit VRAC / PRECOND*
- *Saisonnalité (périodes de disponibilité)*
- *Lots de stock avec DLC / DLUO*
- *Commande → mode paiement → mode récupération*
- *Livraison → poids, frais, date estimée*
- *Gestion des contenants*
- *Rôle épicier (alertes + ajustement des prix + clôture)*





MODÈLE E/A

TRADUCTION RELATIONNELLE

**NOUS AVONS CHOISI DE FORTEMENT CONTRAINDRE LE MODÈLE SQL
(CHECK, DATES, UNITÉS), CE QUI RÉDUIT LES ERREURS CÔTÉ JAVA**

```
CREATE TABLE PRODUIT (  
  IDPRODUIT NUMBER PRIMARY KEY,  
  NOMPRODUIT VARCHAR2(100) NOT NULL,  
  CATEGORIEPRODUIT VARCHAR2(100),  
  DESCRIPT VARCHAR2(300),  
  IDPRODUCTEUR NUMBER NOT NULL,  
  FOREIGN KEY (IDPRODUCTEUR) REFERENCES PRODUCTEUR(IDPRODUCTEUR)  
);
```

```
CREATE TABLE COMMANDE (  
  IDCOMMANDE NUMBER PRIMARY KEY,  
  DATECOMMANDE DATE,  
  HEURECOMMANDE VARCHAR2(12),  
  STATUT VARCHAR2(30),  
  MODEPAIEMENT VARCHAR2(20)  
    CHECK (MODEPAIEMENT IN ('EN LIGNE', 'EN  
BOUTIQUE'))),  
  IDCLIENT NUMBER NOT NULL,  
  FOREIGN KEY (IDCLIENT) REFERENCES CLIENT(IDCLIENT),  
  CONSTRAINT VERIFSTATUT CHECK (  
    STATUT IN ('En preparation', 'Prete', 'En  
livraison', 'Recupere', 'Livree', 'Annulee')  
  ),  
  FOREIGN KEY (STATUT) REFERENCES  
STATUT_COMMANDE(STATUT)  
);
```

```
CREATE TABLE LOT_PRODUIT (  
  IDLOTPRODUIT NUMBER PRIMARY KEY,  
  IDPRODUIT NUMBER NOT NULL,  
  QUANTITESTOCKLOT NUMBER,  
  UNITESTOCKLOT VARCHAR2(20) CHECK (UNITESTOCKLOT  
IN ('Kg', 'Unite')),  
  DATERECEPTION DATE,  
  DATEPEREMPTION DATE,  
  TYPEDATE VARCHAR2(10) NOT NULL CHECK (TYPEDATE IN  
( 'DLC', 'DLUO' )),  
  FOREIGN KEY (IDPRODUIT) REFERENCES  
PRODUIT(IDPRODUIT) ON DELETE CASCADE,  
  CONSTRAINT LOTPRODUITDATE CHECK (DATEPEREMPTION  
>= DATERECEPTION),  
  CONSTRAINT LOTPRODUIT_DLC_DLUO_XOR CHECK (  
    DATEPEREMPTION IS NOT NULL  
    AND TYPEDATE IN ('DLC', 'DLUO')  
  ),  
  -- au plus une livraison par jour et par produit  
  CONSTRAINT LOTPRODUIT_UNQ_JOUR UNIQUE (IDPRODUIT,  
DATERECEPTION)  
);
```

```
CREATE TABLE CONTENANT (  
  IDCONTENANT NUMBER PRIMARY KEY,  
  TYPECONTENANT VARCHAR2(50),  
  CAPACCONTENANT NUMBER,  
  UNITECONTENANT VARCHAR2(10) CHECK (UNITECONTENANT IN  
( 'LITRE', 'GRAMME' )),  
  STOCKDISPOCONTENANT NUMBER,  
  CARACTCONTENANT VARCHAR2(50) CHECK (CARACTCONTENANT IN  
( 'JETABLE', 'REUTILISABLE' )),  
  PRIXVENTECONTENANT NUMBER  
);
```

TRANSACTIONS & CONCURRENCE

UNE COMMANDE = UNE TRANSACTION COMPLÈTE

- **Gestion manuelle JDBC** : `setAutoCommit(false)`, `commit()`, `rollback()`
- **Verrouillage pessimiste** : `SELECT ... FOR UPDATE` pour réserver les lots
- **Déstockage sécurisé** : `UPDATE LOT_PRODUIIT SET stock = stock - ? WHERE stock >= ?`
- **Aucun stock négatif possible**
- **FIFO sur les lots** (tri par date de péremption)
- **Toute la F1 s'exécute dans une seule transaction**

FONCTIONNALITÉ F1 : PASSAGE DE COMMANDE

**LE PASSAGE DE COMMANDE EST LA FONCTIONNALITÉ LA PLUS COMPLEXE :
UTILISE TOUTES LES CONTRAINTES SQL ET TOUTES LES TRANSACTIONS**

- 1. Vérification client**
- 2. Choix mode récupération**
- 3. Saisie produits (VRAC / PRECOND)**
- 4. Gestion des contenants**
- 5. Vérifications : Saisonnalité, Stock total disponible**
- 6. Calcul prix & poids**
- 7. Déstockage lot-par-lot (verrouillé)**
- 8. Commit final**



FONCTIONNALITÉ F2 : ALERTE & RÉDUCTION

- Détection des lots périssables (< 7 jours)
- Affichage épicier
- Réduction automatique des prix :
- VRAC → baisse **PRIXVENTEVRAC**
- PRECOND → baisse **PRIXVENTEP**
- Transaction encapsulée (commit/rollback)

FONCTIONNALITÉ F3 : CLÔTURE DE COMMANDE

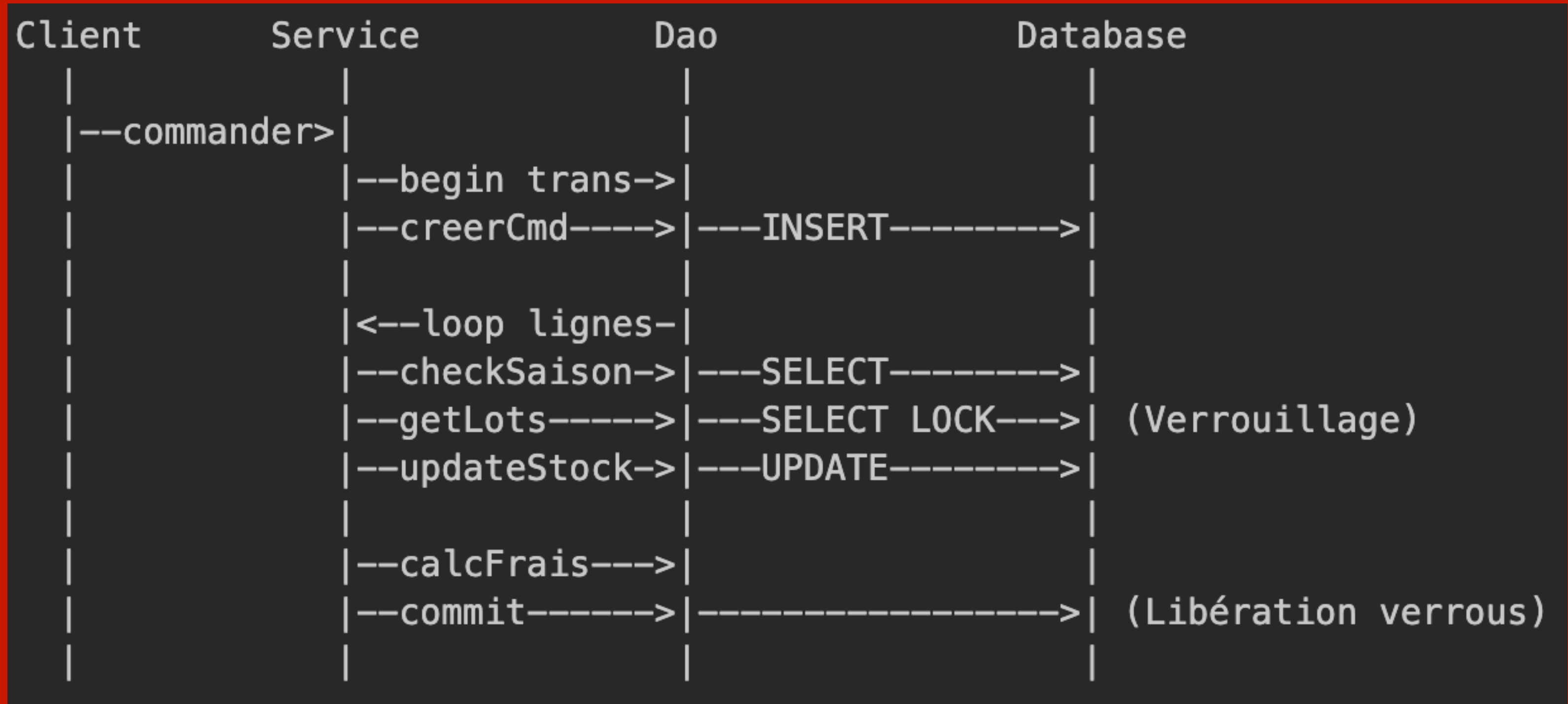
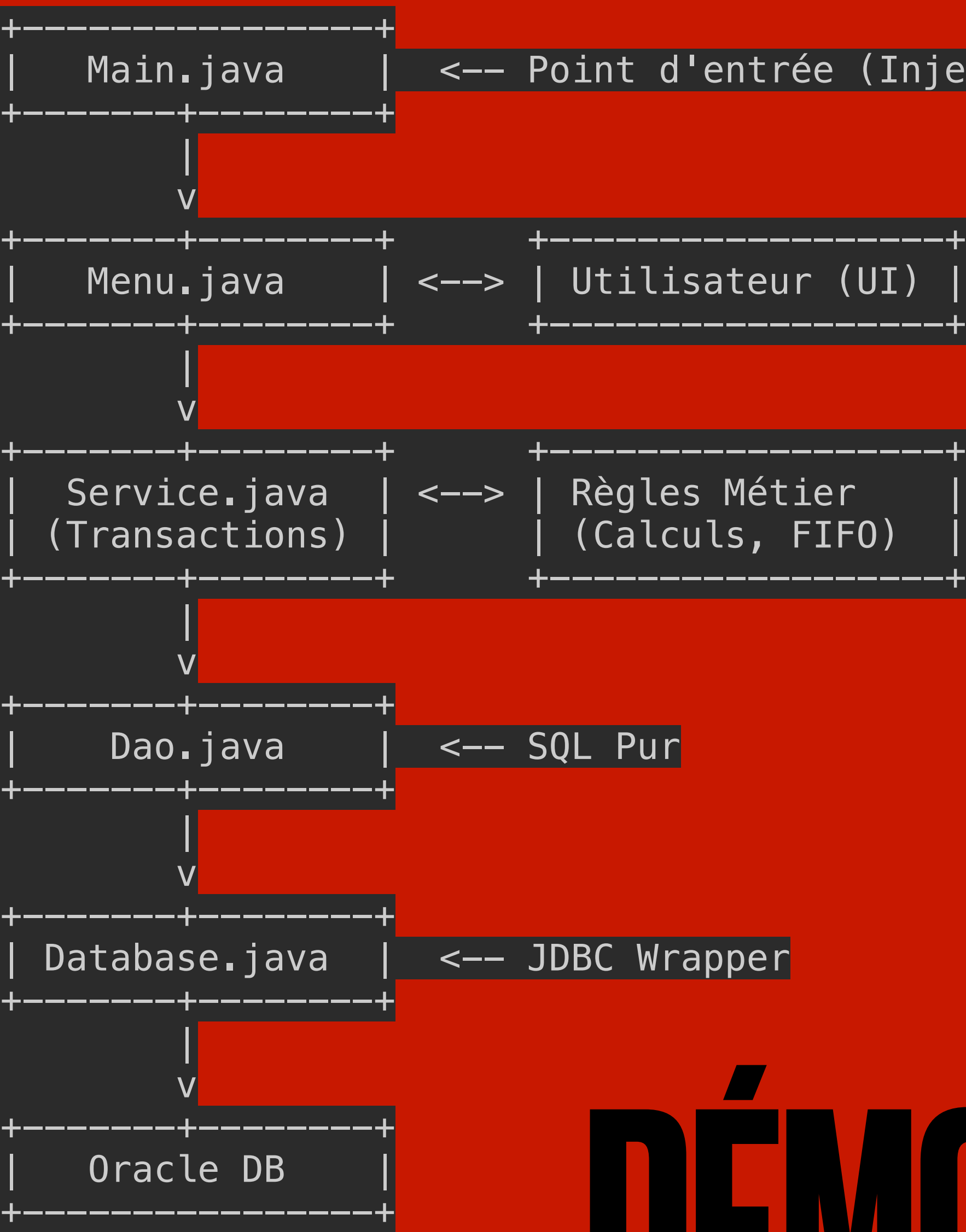
USAGE DES TRANSACTIONS POUR SYNCHRONISER MISE À JOUR ET STATUT

- Verrouillage de la commande : FOR UPDATE
- Vérification que le statut est clôturable
- Mise à jour :
- Date réelle
- Statut final → Récupérée / Livrée
- Commit final

```
private void cloturerCommande() {
    try {
        System.out.print("ID
commande : ");
        int id =
Integer.parseInt(sc.nextLine());

        service.cloturerCommande(id);
        System.out.println("Waaw,
Commande clôturée.");
    } catch (Exception e) {
        System.out.println("Erreur
de Clôture : " + e.getMessage());
    }
}
```

```
public void cloturerCommande(int idCommande) throws Exception {
    db.setAutoCommit(false);
    try {
        String statut =
dao.getStatutEtVerrouillerCommande(idCommande);
        List<String> etatsPossibles = List.of("En preparation",
"Prete", "En livraison");
        if (!etatsPossibles.contains(statut)) {
            throw new Exception("Commande non clôturable au statut : "
+ statut);
        }
        String[] modes = dao.getModeRecupAndPaieement(idCommande);
        String modeRecup = modes[0];
        String nouveauStatut = modeRecup.equals("Retrait") ?
"Recupere" : "Livree";
        dao.enregistrerDateRecup(idCommande, modeRecup);
        dao.updateStatutCommande(idCommande, nouveauStatut);
        db.commit();
    } catch (Exception e) {
        db.rollback();
        throw new Exception("Erreur clôture : " + e.getMessage());
    } finally {
        db.setAutoCommit(true);
    }
}
```

DÉMONSTRATEUR JAVA

ORGANISATION DE L'ÉQUIPE

A. • UML + analyse → Tous (travail collaboratif pour valider la compréhension)

B. • Modèle relationnel [Pivot]

C. • Implémentation SQL [Documentation]

D. • Démonstrateur Java [Java + Slides]

E. • Fonctionnalités métier [Coordinateur]